

目录

- TOC {toc}

我们的下一个并行模式是前缀和(prefix sum)，它通常也被称为扫描(scan)。并行扫描经常用于并行化看似顺序的操作，例如资源分配、工作分配和多项式求值(polynomial evaluation)。一般来说，如果某个计算自然地描述为一个数学递归，其中系列中的每一项都以之前的项为基础定义，那么它很可能可以并行化为一个并行扫描操作。并行扫描在大规模并行计算中起着关键作用，其原因很简单：应用程序中的任何顺序部分都可能极大地限制整个应用程序的性能。许多此类顺序部分可以通过并行扫描转换为并行计算。因此，并行扫描常被用作并行算法中的基本操作，这些算法包括基数排序、快速排序、字符串比较、多项式求值、求解递归、树操作和流压缩(stream compaction)。基数排序的例子将在第十三章《排序》中介绍。

另一个并行扫描是重要并行模式的原因是，它是某些并行算法执行的工作复杂度可能比顺序算法高的典型例子，这导致了在算法复杂性和并行化之间需要谨慎权衡。正如我们将展示的那样，算法复杂性的略微增加可能使并行扫描在处理大数据集时运行速度比顺序扫描慢。在“大数据”时代，这种考虑变得更加重要，因为海量数据集对具有高计算复杂度的传统算法提出了挑战。

11.1 背景

从数学上讲，一个包含的(inclusive)扫描操作采用一个二元结合运算符 $\oplus$ 和一个包含 n 个元素的输入数组 $[x_0, x_1, \dots, x_{n-1}]$ ，并返回以下输出数组：

$$[x_0, (x_0 \oplus x_1), \dots, (x_0 \oplus x_1 \oplus \dots \oplus x_{n-1})]$$

例如，如果运算符 $\oplus$ 是加法，那么对输入数组 $[3, 1, 7, 0, 4, 1, 6, 3]$ 进行包含扫描操作将返回 $[3, 3+1, 3+1+7, 3+1+7+0, \dots, 3+1+7+0+4+1+6+3] = [3, 4, 11, 11, 15, 16, 22, 25]$ 。名称“包含”扫描来源于每个输出元素都包含了相应输入元素的效果。

我们可以通过一个为一组人切香肠的例子来说明包含扫描操作的应用。假设我们有一根40英寸长的香肠要分给八个人。每个人要求的香肠长度（以英寸为单位）各不相同：3, 1, 7, 0, 4, 1, 6, 和 3。也就是说，第0个人需要3英寸香肠，第1个人需要1英寸，依此类推。我们可以顺序地或并行地切香肠。顺序的方法非常简单。我们先为第0个人切一段3英寸的香肠，此时香肠还剩37英寸。然后为第1个人切一段1英寸的香肠，香肠变为36英寸长。我们可以继续切更多的段，直到为第7个人切出3英寸的香肠。此时，我们总共切出了25英寸的香肠，还剩15英寸。

通过包含扫描操作，我们可以根据每个人订购的数量计算所有的切割位置。也就是说，给定加法操作和订单输入数组 $[3, 1, 7, 0, 4, 1, 6, 3]$ ，包含扫描操作返回 $[3, 4, 11, 11, 15, 16, 22, 25]$ 。返回数组中的数字就是切割位置。有了这些信息，可以同时进行所有的八次切割，以生成每个人订购的段。第一个切割点在3英寸位置，所以第一个段是3英寸长，正如第0个人订购的那样。第二个切割点在4英寸位置，因此第二段是1英寸长，正如第1个人订购的那样。最后一个切割点在25英寸位置，由于之前的切割点在22英寸位置，这将产生一个3英寸长的段，正如第7个人订购的那样。注意，由于所有的切割点都通过扫描操作得知，所以所有切割可以并行进行或按任何顺序进行。

总之，一种直观的思考包含扫描操作的方法是，这个操作接收一组人的请求，并确定所有切割点，使得可以一次性满足所有的订单。订单可以是香肠、面包、露营地空间，或者计算机中的连续内存块。只要我们能快速计算出所有的切割点，所有订单就可以并行满足。

排他扫描操作与包含扫描操作类似，但输出数组的排列略有不同：

$$[i, x_0, (x_0 \oplus x_1), \dots, (x_0 \oplus x_1 \oplus \dots \oplus x_{n-2})]$$

也就是说，每个输出元素不包括相应输入元素的影响。第一个输出元素是 i ，即运算符的单位元，而最后一个输出元素只反映到 x_{n-2} 的贡献。二元运算符的单位元被定义为一种值，当它作为输入操作数时，使操作生成的输出值与另一个输入操作数的值相同。在加法运算的情况下，单位元是 0，因为任何数加上零的结果都是它本身。

排斥扫描操作的应用与包含扫描操作几乎相同。包含扫描提供略微不同的信息。在香肠的例子中，排斥扫描将返回 [0 3 4 11 11 15 16 22]，这些是切割段的起始点。例如，第0个人的段从0英寸点开始。再比如，第7个人的段从22英寸点开始。起始点信息在诸如内存分配等应用中非常重要，在这些应用中，分配的内存通过指向其起始点的指针返回给请求者。

注意，在包含扫描输出和排斥扫描输出之间的转换很容易。只需将所有元素移动并填充一个元素即可。从包含扫描转换为排斥扫描时，只需将所有元素向右移动，并为第0个元素填充单位元。从排斥扫描转换为包含扫描时，需要将所有元素向左移动，并用最后一个输入元素加上最后一个输出元素的结果填充最后一个元素。这只是为了方便，我们可以直接生成包含或排斥扫描，具体取决于我们是否关心切割点或段的起始点。因此，我们将只为包含扫描提供并行算法和实现。

在展示并行扫描算法及其实现之前，我们先展示一个顺序包含扫描算法及其实现。我们假设涉及的运算符是加法。图11.1中的代码假设输入元素在 x 数组中，输出元素将写入 y 数组中。代码将输出元素 $y[0]$ 初始化为输入元素 $x[0]$ 的值（第02行）。在循环的每次迭代中（第03-05行），循环体将另一个输入元素添加到前一个输出元素（其存储所有先前输入元素的累加）中以生成另一个输出元素。

```
01 void sequential_scan(float *x, float *y, unsigned int N) {
02     y[0] = x[0];
03     for(unsigned int i = 1; i < N; ++i) {
04         y[i] = y[i - 1] + x[i];
05     }
06 }
```

图11.1 基于加法的简单顺序包含扫描实现。

显然，图11.1中顺序实现包含扫描的工作量与输入元素的数量成线性比例；即顺序算法的计算复杂度为 $O(N)$ 。在第11.2-11.5节中，我们将展示执行并行分段扫描的替代算法，其中每个线程块将对输入数组中的一个段，即一个部分的元素进行并行扫描。然后我们将在第11.6和11.7节中展示将分段扫描结果合并为整个输入数组的扫描输出的方法。

11.2 使用 Kogge-Stone 算法的并行扫描

我们从一个简单的并行包含扫描算法开始，通过为每个输出元素执行归约操作。有人可能会想用每个线程为一个输出元素执行顺序归约，如图10.2所示。毕竟，这允许所有输出元素的计算并行进行。不幸的是，这种方法不太可能提高图11.1中的顺序扫描代码的执行时间。这是因为计算 y_{n-1} 需要 n 步，与顺序扫描代码所需的步数相同，而且归约中的每一步（迭代）所涉及的工作量与顺序扫描的每次迭代相同。由于并行程序的完成时间受耗时最长的线程限制，这种方法不太可能比顺序扫描更快。事实上，在计算资源有限的情况下，这种简单的并行扫描算法的执行时间可能比顺序算法长得多。不幸的是，所提出方法的计算成本，即执行的总操作数会大大增加。由于输出元素 i 的归约步骤数为 i ，所有线程执行的总步骤数将为：

$$\sum_{i=0}^{n-1} i = \frac{n \cdot (n-1)}{2}$$

也就是说，所提出的方法的计算复杂度为 $O(N^2)$ ，比顺序扫描的复杂度 $O(N)$ 高，但并未提供任何加速效果。更高的计算复杂度意味着需要配置大量的执行资源。这显然是个坏主意。

更好的方法是改编第10章《归约与最小化分歧》中的并行归约树，使用相关输入元素的归约树计算每个输出元素。有多种设计每个输出元素归约树的方法。由于元素 i 的归约树涉及 i 次加法操作，除非我们找到一种在不同输出元素的归约树之间共享部分和的方法，否则这种方法仍会将计算复杂度提高到 $O(N^2)$ 。我们提出了一种基于 Kogge-Stone 算法的共享方法，该算法最初是在20世纪70年代为设计快速加法器电路而发明的 (Kogge & Stone, 1973)。该算法仍然用于设计高速计算机算术硬件。

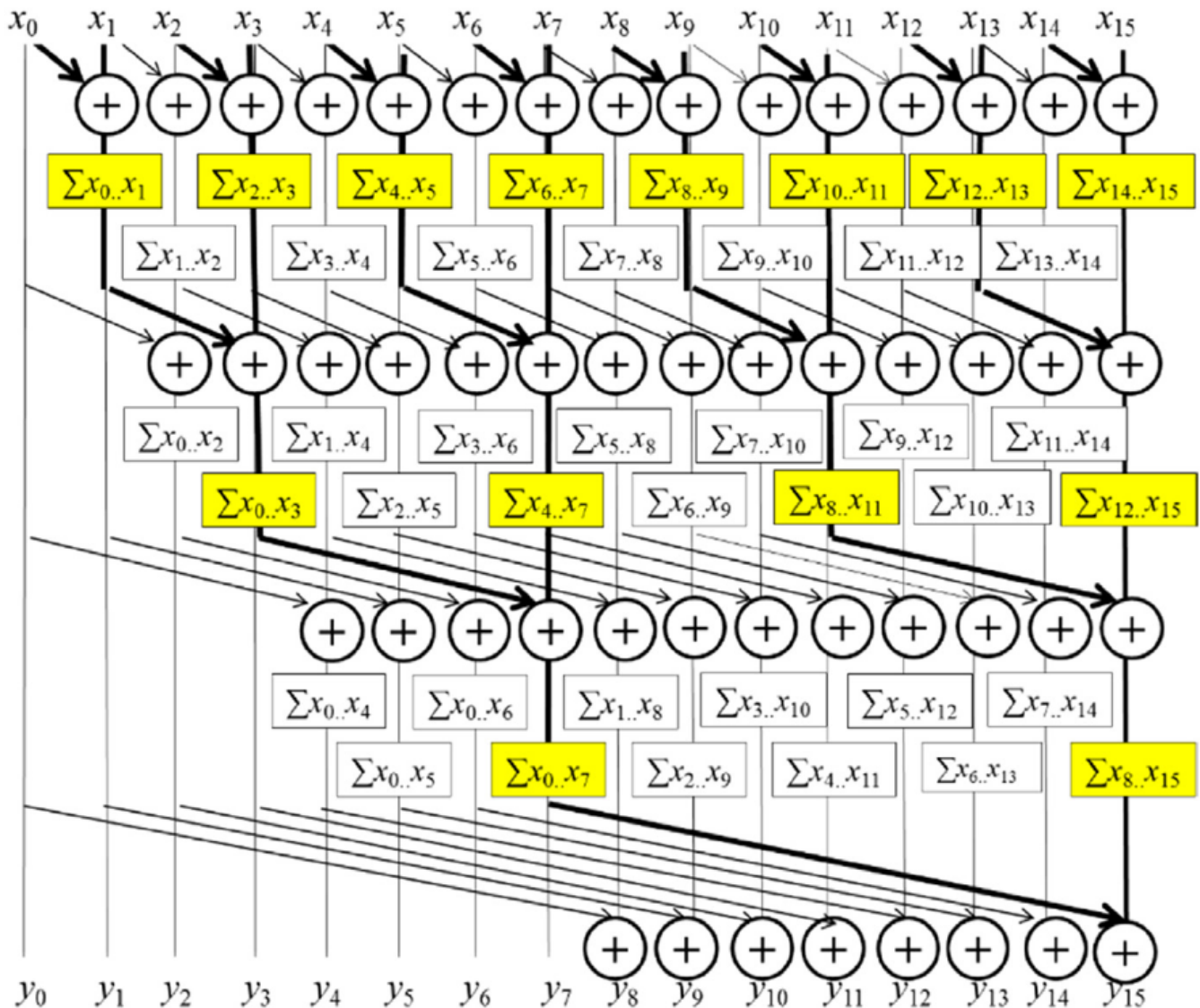


图11.2 基于Kogge-Stone加法器设计的并行包含扫描算法。

该算法如图11.2所示，是一种就地扫描算法，对最初包含输入元素的数组XY进行操作。它通过迭代逐步将数组的内容转换为输出元素。在算法开始之前，我们假设 $XY[i]$ 包含输入元素 x_i 。经过 k 次迭代后， $XY[i]$ 将包含最多 2^k 个输入元素的和，位于当前位置及之前。例如，经过一次迭代后， $XY[i]$ 将包含 $x_{i-1} + x_i$ ；在第2次迭代结束时， $XY[i]$ 将包含 $x_{i-3} + x_{i-2} + x_{i-1} + x_i$ ，以此类推。

图11.2用一个包含16个元素的输入示例说明了该算法。每条垂直线代表XY数组的一个元素，最左边的位置为 $XY[0]$ 。垂直方向显示了迭代的进展，从图的顶部开始。根据包含扫描的定义， y_0 为 x_0 ，因此 $XY[0]$ 包含其最终结果。在第一次迭代中，除 $XY[0]$ 外的每个位置都接收其当前内容和左侧邻居的和。这在图11.2的第一行加法运算符中有所体现。结果是， $XY[i]$ 包含 $x_{i-1} + x_i$ 。这反映在图11.2第一行加法运算符下方的标记框中。例如，在第一次迭代后， $XY[3]$ 包含 $x_2 + x_3$ ，表示为 $\sum_{x_2..x_3}$ 。注意，第一次迭代后， $XY[1]$ 等于 $x_0 + x_1$ ，这是该位置的最终结果。因此，在随后的迭代中不应再对 $XY[1]$ 进行更改。

在第二次迭代中，除XY[0]和XY[1]外的每个位置都接收其当前内容和距离其两个元素远的位置的和。这在第二行加法运算符下方的标记框中有所体现。结果是，XY[i]变为 $x_{i-3} + x_{i-2} + x_{i-1} + x_i$ 。例如，第二次迭代后，XY[3]变为 $x_0 + x_1 + x_2 + x_3$ ，表示为 $\sum_{x_0 \dots x_3}$ 。注意，第二次迭代后，XY[2]和XY[3]已达到其最终结果，在随后的迭代中不再需要更改。读者可以继续研究剩余迭代的过程。

图11.3显示了图11.2所示算法的并行实现。我们实现了一个核函数，该函数在输入的不同段（部分）上执行局部扫描，每个段都小到足够由一个块处理。稍后，我们将进行最终调整，以整合这些部分扫描结果用于大型输入数组。段的大小定义为编译时常量SECTION_SIZE。我们假设将使用SECTION_SIZE作为块大小调用核函数，因此线程数与段元素数相同。我们为每个线程分配一个XY元素内容的演化任务。

```

01  __global__ void Kogge_Stone_scan_kernel(float *X, float *Y, unsigned int N){
02      __shared__ float XY[SECTION_SIZE];
03      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
04      if(i < N) {
05          XY[threadIdx.x] = X[i];
06      } else {
07          XY[threadIdx.x] = 0.0f;
08      }
09      for(unsigned int stride = 1; stride < blockDim.x; stride *= 2) {
10          __syncthreads();
11          float temp;
12          if(threadIdx.x >= stride)
13              temp = XY[threadIdx.x] + XY[threadIdx.x-stride];
14          __syncthreads();
15          if(threadIdx.x >= stride)
16              XY[threadIdx.x] = temp;
17      }
18      if(i < N) {
19          Y[i] = XY[threadIdx.x];
20      }
21  }

```

图11.3 用于包含（分段）扫描的Kogge-Stone核函数。

图11.3中展示的实现假设输入值最初位于全局内存数组X中，并将其地址作为参数传递给核函数（第1行）。我们将块中的所有线程协作地将X数组元素加载到共享内存数组XY中（第2行）。这是通过使每个线程计算其全局数据索引 $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ （第3行）来完成的，以负责其对应的输出向量元素位置。每个线程在核函数开始时将该位置的输入元素加载到共享内存中（第4-8行）。在核函数结束时，每个线程将其结果写入分配的输出数组Y中（第18-20行）。

现在我们将重点放在图11.3中每个XY元素的迭代计算的实现上，这是一个for循环（第9-17行）。该循环迭代了分配给一个线程的XY数组位置的归约树。当步幅值大于一个线程的threadIdx.x值时，意味着该线程分配的XY位置已经累积了所有所需的输入值，线程不再需要活跃（第12和15行）。注意，我们使用了屏障同步（第10行），以确保所有线程在开始下一个迭代之前完成了上一个迭代。这与第10章《归约与最小化分歧》中对归约讨论中__syncthreads()的使用相同。

然而，与归约相比，在for循环的每次迭代中更新XY元素（第12-16行）方面存在非常重要的区别。请注意，每个活跃线程首先将其位置的部分和存储到一个临时变量中（寄存器中）。在所有线程完成第二个屏障同步后（第14行），它们都将部分和值存储到其XY位置（第16行）。额外的temp和__syncthreads()的需求与这些更新中的读后写数据依赖性危险有关。每个活跃线程将其自己位置（XY[threadIdx.x]）和另一个线程位置

（XY[threadIdx.x-stride]）的XY值相加。如果一个线程i在另一个线程i+stride有机会读取该位置的旧值之前写入其输出位置，则新值可能会破坏其他线程执行的加法。这种破坏可能发生也可能不发生，这取决于涉及线程的执行时间，这被称为竞态条件。请注意，这种竞态条件与我们在第9章《并行直方图》中看到的不同。第9章《并行直方图》中的竞态条件是一种读取-修改-写入的竞态条件，可以通过原子操作来解决。对于我们在这里看到的读后写竞态条件，需要采用不同的解决方案。

在图11.2中可以很容易地观察到竞态条件。让我们来检查迭代2中线程4 (x4) 和线程6 (x6) 的活动，这在从顶部开始的第二行加法操作中表示。请注意，线程6需要将XY[4]的旧值 (x3+x4) 与XY[6]的旧值 (x5+x6) 相加，以生成XY[6]的新值 (x3 + x4 + x5 + x6)。然而，如果线程4在迭代中太早将其加法结果 (x1+x2+x3+x4) 存储到XY[4]中，线程6可能会使用新值作为其输入并将(x1+x2 + x3+x4+x5+x6)存储到XY[6]中。由于线程6在第三次迭代中会再次将x1+x2添加到XY[6]，XY[6]的最终答案将变为(2x1 + 2x2+x3+x4+x5 + x6)，这显然是不正确的。另一方面，如果在线程4在迭代2期间覆盖之前，线程6恰好读取XY[4]的旧值，则结果将是正确的。也就是说，代码的执行结果可能正确也可能不正确，这取决于线程执行的时间，执行结果可能因运行而异。这种缺乏可重现性可能会使调试变成一场噩梦。

通过在第13行使用临时变量和第14行的__syncthreads()屏障，可以解决竞态条件。在第13行，所有活跃线程首先执行加法并写入其私有的临时变量。因此，不会覆盖XY位置的旧值。第14行中的屏障__syncthreads()确保所有活跃线程在任何一个线程继续执行写操作之前都完成了对旧XY值的读取。因此，第16行的语句可以安全地覆盖XY位置。

更新的XY位置可能被另一个活跃线程使用的原因是，Kogge-Stone方法通过在不同输出元素之间重复使用部分和来减少计算复杂性。我们将在第11.3节进一步研究这一点。读者可能会想知道为什么第10章《归约与最小化分歧》中的归约树核心不需要使用临时变量和额外的__syncthreads()。答案是，这些归约核心中没有由写后读危险引起的竞态条件。这是因为在一个迭代中，由活跃线程写入的元素不会被任何其他活跃线程读取。通过检查图10.7和10.8，这一点应该是显而易见的。例如，在图10.8中，每个活跃线程从其自己的位置

(input[threadIdx.x]) 和距离右边stride距离的位置 (input[threadIdx.x+stride]) 获取其输入。在任何给定的迭代中，任何一个活跃线程都不会更新距离stride位置。因此，所有活跃线程将始终能够读取其相应input[threadIdx.x]的旧值。由于线程内的执行始终是顺序的，每个线程将始终能够在将新值写入位置之前读取input[threadIdx.x]的旧值。读者应该验证相同的属性在图10.7中是否成立。

如果我们想要在每次迭代中避免第二个屏障同步，另一种克服竞态条件的方法是使用单独的数组来存储输入和输出。如果使用单独的数组，正在写入的位置与正在读取的位置不同，因此不再存在任何可能的写后读竞态条件。这种方法需要两个共享内存缓冲区而不是一个。在开始时，我们从全局内存加载到第一个缓冲区。在第一次迭代中，我们从第一个缓冲区读取并写入第二个缓冲区。在迭代结束后，第二个缓冲区具有最新的结果，第一个缓冲区中的结果不再需要。因此，在第二次迭代中，我们从第二个缓冲区读取并写入第一个缓冲区。按照同样的逻辑，第三次迭代中我们从第一个缓冲区读取并写入第二个缓冲区。我们继续交替使用输入/输出缓冲区，直到迭代完成。这种优化称为双缓冲。双缓冲在并行编程中常用作克服写后读竞态条件的一种方式。我们将这种优化的实现留给读者作为练习。

此外，如图11.2所示，对XY较小位置的操作会比较早结束 (参见if语句的条件)。当步长值较小时，这将导致第一个warp中存在一定程度的控制分歧。请注意，相邻的线程往往会执行相同数量的迭代。对于大块大小，分歧的影响应该相当有限，因为分歧只会出现在第一个warp中。详细的分析留给读者作为练习。

尽管我们只展示了一个包含扫描的核心，但我们可以轻松地将包含扫描核心转换为排他扫描核心。回想一下，排他扫描等同于包含扫描，只是所有元素向右移动一个位置，并且元素0被填充为单位元。这在图11.4中有所体现。请注意，唯一的真正区别是图顶部元素的对齐方式。所有标记框都已更新以反映新的对齐方式。所有迭代操作保持不变。

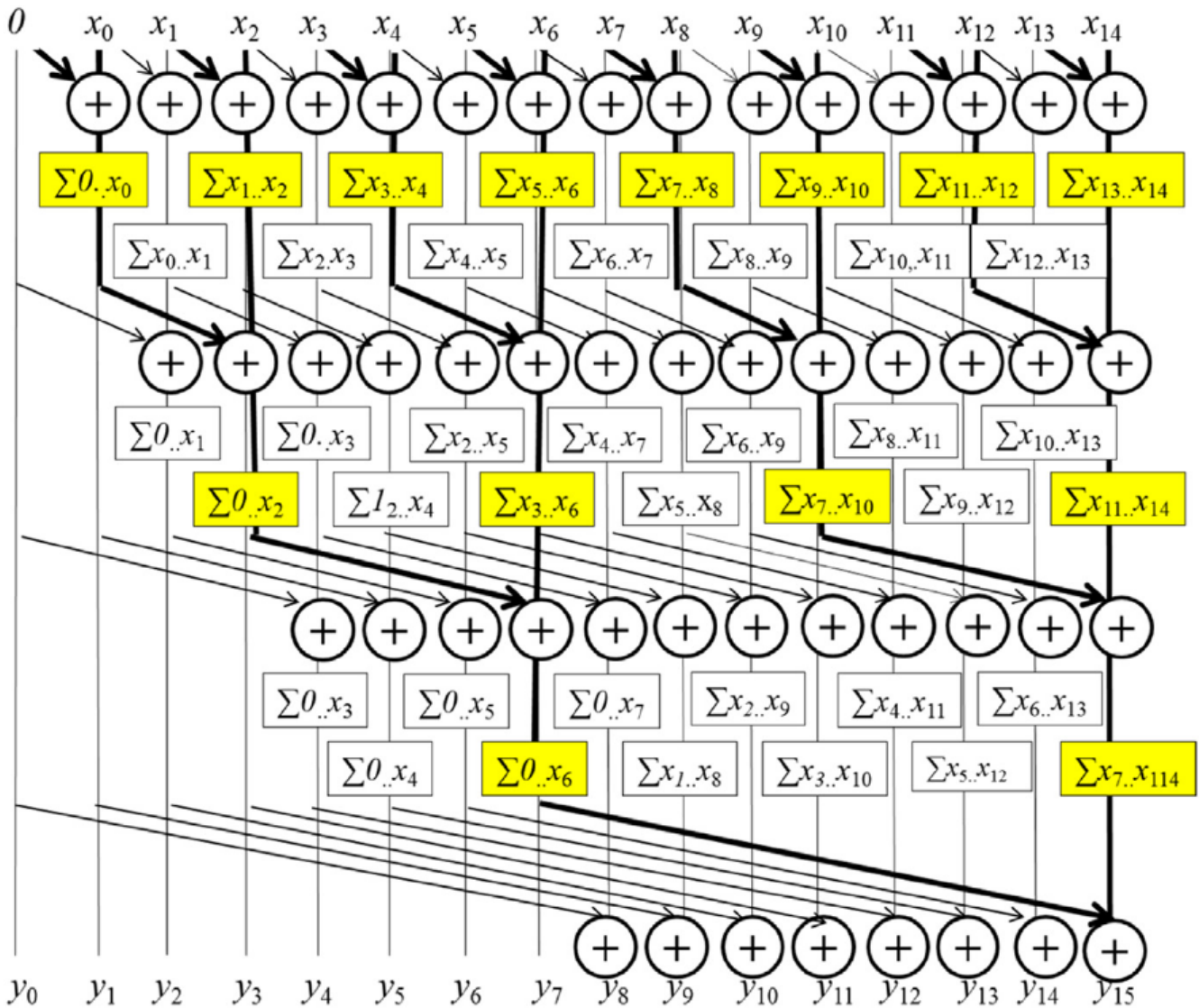


图11.4 用于包含 (分段) 扫描的Kogge-Stone核函数。

现在我们可以轻松地将图11.3中的核心转换为排他扫描核心。我们需要做的唯一修改是将0加载到XY[0]中，并将X[i-1]加载到XY[threadIdx.x]中，代码如下所示：

```
if (i < N && threadIdx.x != 0) {
    XY[threadIdx.x] = X[i-1];
} else {
    XY[threadIdx.x] = 0.0f;
}
```

通过用这四行代码替换图11.3的04至08行，我们将包含扫描核心转换为排他扫描核心。完成排他扫描核心的工作留给读者作为练习。

11.3 速度和工作效率考虑

分析并行算法时，一个重要的考虑因素是工作效率。算法的工作效率是指算法执行的工作量与计算所需的最小工作量的接近程度。例如，扫描操作所需的最小加法次数是 $N-1$ 次，或 $O(N)$ ，这是顺序算法执行的加法次

数。然而，如我们在11.2节开头所看到的，朴素的并行算法执行了 $N*(N-1)/2$ 次加法，或 $O(N^2)$ ，这明显比顺序算法多得多。因此，朴素的并行算法不是工作高效的。

我们现在分析图11.3中的Kogge-Stone内核的工作效率，重点是单个线程块的工作。所有线程最多迭代 $\log_2 N$ 步，其中 N 是 `SECTION_SIZE`。在每次迭代中，非活动线程的数量等于步长大小。因此，我们可以计算出算法的工作量（for 循环的一次迭代，用图8.1中的加法操作表示）为：

$$\sum_{\text{stride}} (N - \text{stride}) \quad \text{stride} = 1, 2, 4, \dots, N/2$$

每项的第一部分与步长无关，其总和为 $N * \log_2(N)$ 。第二部分是熟悉的几何级数，总和为 $(N - 1)$ 。因此，总工作量为：

$$N * \log_2 N - (N - 1)$$

好消息是，Kogge-Stone方法的计算复杂度是 $N * \log_2(N)$ ，优于执行完整归约树以处理所有输出元素的朴素方法的 $O(N^2)$ 复杂度。坏消息是，Kogge-Stone算法的工作效率仍然不如顺序算法。即使对于中等大小的区段，图11.3中的内核也比顺序算法多做了很多工作。在512个元素的情况下，该内核执行的工作量约为顺序代码的八倍。随着 N 增大，这一比例会增加。

虽然Kogge-Stone算法比顺序算法执行更多的计算，但由于并行执行，它在更少的步骤中完成这些计算。顺序代码的 for 循环执行 N 次迭代。对于内核代码，每个线程的 for 循环最多执行 $\log_2 N$ 次迭代，这是执行内核所需的最少步骤。在无限执行资源的情况下，内核代码相对于顺序代码的步骤减少量大约为 $N / \log_2(N)$ 。对于 $N=512$ ，步骤减少量大约为 $512/9=56.93$ 。

在实际的 CUDA GPU 设备中，Kogge-Stone内核的工作量比理论上的 $N * \log_2 N - (N - 1)$ 多。这是因为我们使用了 N 个线程。虽然许多线程停止参与 for 循环的执行，但一些线程仍然消耗执行资源，直到整个 warp 完成执行。现实中，Kogge-Stone 的执行资源消耗更接近于 $N * \log_2 N$ 。

我们将使用计算步骤的概念作为比较扫描算法的近似指标。顺序扫描处理 N 个输入元素大约需要 N 步。例如，顺序扫描处理 1024 个输入元素大约需要 1024 步。使用 CUDA 设备中的 P 个执行单元，我们可以期望 Kogge-Stone 内核执行 $N * \log_2 N / P$ 步。如果 P 等于 N ，即我们有足够的执行单元来并行处理所有输入元素，那么我们需要 $\log_2 N$ 步，如前所述。然而， P 可能小于 N 。例如，如果我们使用 1024 个线程和 32 个执行单元来处理 1024 个输入元素，内核可能需要 $(1024 * 10)/32 = 320$ 步。在这种情况下，我们预计步骤数量减少 $1024/320 = 3.2$ 倍。

Kogge-Stone 内核相对于顺序代码的额外工作在两方面存在问题。首先，硬件执行并行内核的效率大大降低。如果硬件资源不足（即 P 较小），并行算法可能需要比顺序算法更多的步骤。因此，并行算法会更慢。其次，所有额外的工作消耗了更多的能量。这使得内核不太适合功率受限的环境，例如移动应用。

Kogge-Stone 内核的优势在于，当有足够的硬件资源时，它可以实现非常好的执行速度。它通常用于计算具有适中元素数量的区段的扫描结果，例如 512 或 1024。当然，这假设 GPU 可以提供足够的硬件资源并利用额外的并行性来容忍延迟。正如我们所看到的，它的执行有非常有限的控制分歧。在更新的 GPU 架构中，其计算可以通过 warp 内的 shuffle 指令高效完成。稍后在本章中我们将看到它是现代高速并行扫描算法的重要组成部分。

11.4 使用 Brent-Kung 算法的并行扫描

虽然图 11.3 中的 Kogge-Stone 内核在概念上很简单，但对于某些实际应用来说，其工作效率相当低。仅通过查看图 11.2 和 11.4，我们可以看到有进一步共享一些中间结果的潜在机会。然而，为了允许多个线程之间更多的共享，我们需要战略性地计算中间结果并将其分配给不同的线程，这可能需要额外的计算步骤。

我们知道，生成一组值的和最快的并行方式是使用归约树。在有足够执行单元的情况下，归约树可以在 $\log_2(N)$ 时间单位内生成 N 个值的和。归约树还可以生成几个子和，这些子和可用于计算一些扫描输出值。这一观察结果被用作 Kogge-Stone 加法器设计的基础，也构成了 Brent-Kung 加法器设计的基础（Brent & Kung, 1979）。Brent-Kung 加法器设计也可以用来实现工作效率更高的并行扫描算法。

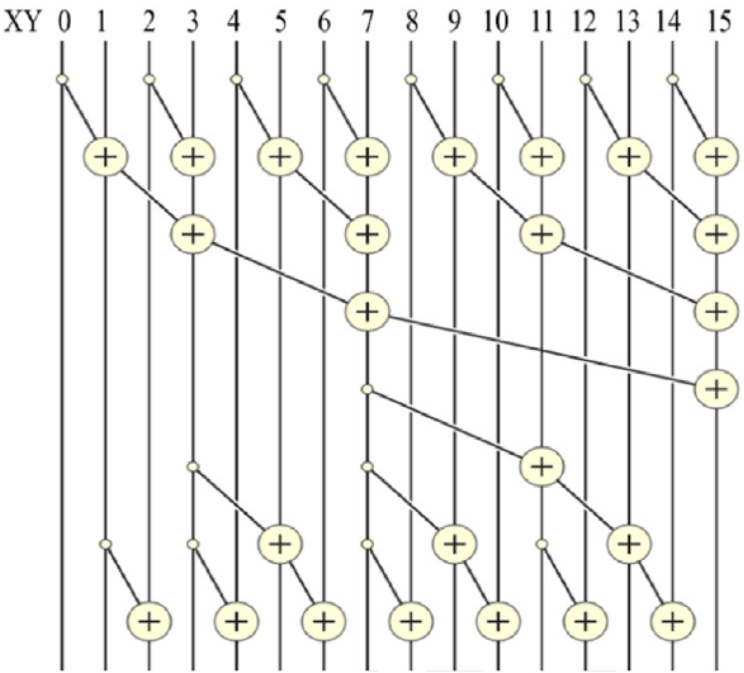


图11.5 基于 Brent-Kung 加法器设计的并行包含扫描算法。

图 11.5 说明了基于 Brent-Kung 加法器设计的并行包含扫描算法的步骤。在图 11.5 的上半部分，我们在四个步骤中生成了所有 16 个元素的和。我们使用了生成和所需的最小操作次数。在第一步中，只有 $XY[i]$ 的奇数元素会更新为 $XY[i-1] + XY[i]$ 。在第二步中，只有索引为 $4 * n - 1$ 的 XY 元素会被更新，图 11.5 中的索引为 3、7、11 和 15。在第三步中，只有索引为 $8 * n - 1$ 的 XY 元素会被更新，图 11.5 中的索引为 7 和 15。最后，在第四步中，只有 $XY[15]$ 会被更新。执行的总操作次数为 $8+4+2+1=15$ 。通常，对于包含 N 个元素的扫描区段，我们在此归约阶段会执行 $(N/2) + (N/4) + \dots + 2 + 1 = N - 1$ 次操作。

算法的第二部分是使用反向树将部分和分配到可以使用它们完成这些位置结果的位置。部分和的分配如图 11.5 的下半部分所示。要理解反向树的设计，我们首先需要分析完成 XY 每个位置扫描输出所需的剩余(additional)值。从图 11.5 中可以看出，归约树中的加法总是累加连续范围内的输入元素。因此，我们知道累加到每个 XY 位置的值总是可以表示为输入元素 $x_i \dots x_j$ 的范围，其中 x_i 是起始位置， x_j 是结束位置（包含）。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Initial	x_0	$x_0..x_1$	x_2	$x_0..x_3$	x_4	$x_4..x_5$	x_6	$x_0..x_7$	x_8	$x_8..x_9$	x_{10}	$x_8..x_{11}$	x_{12}	$x_{12}..x_{13}$	x_{14}	$x_0..x_{15}$
Level 1												$x_0..x_{11}$				
Level 2						$x_0..x_5$				$x_0..x_9$				$x_0..x_{13}$		
Level 3			$x_0..x_2$		$x_0..x_4$		$x_0..x_6$		$x_0..x_8$		$x_0..x_{10}$		$x_0..x_{12}$		$x_0..x_{14}$	

图11.6 反向树中每一层加法后 XY 中值的进展情况。

图 11.6 显示了每个位置（列）的状态，包括在反向树的每一层（行）中，已经累积到该位置的值以及每个位置所需的剩余输入元素值。每个位置的状态最初以及在反向树中每一层加法之后，均表示为已经计入该位置的输

入元素形式 $x_i \dots x_j$ 。例如，行 Initial 和列 11 中的 $x_8 \dots x_{11}$ 表示在反向树开始之前（在图 11.5 下半部分的归约阶段之后）， x_8, x_9, x_{10}, x_{11} 的值已累积到 $XY[11]$ 。在归约树阶段结束时，我们有许多位置已完成最终扫描值。在我们的例子中， $XY[0]$ 、 $XY[1]$ 、 $XY[3]$ 、 $XY[7]$ 和 $XY[15]$ 都已完成其最终答案。

每个单元格的阴影表示剩余输入元素值的需求程度：白色表示该位置需要累积三个其他位置的部分和，浅灰色表示需要两个，深灰色表示需要一个，黑色表示不需要。例如，最初 $XY[14]$ 标记为白色，因为在归约树阶段结束时它只有 x_{14} 的值，需要累积 $XY[7]$ ($x_0 \dots x_7$)、 $XY[11]$ ($x_8 \dots x_{11}$) 和 $XY[13]$ ($x_{12} \dots x_{13}$) 的部分和，以完成其最终扫描值 ($x_0 \dots x_{14}$)。读者应验证，由于归约树的结构，对于大小为 N 个元素的输入， XY 位置所需累积的部分和数量从不超过 $\log_2(N) - 1$ 个部分和。此外，这些部分和位置总是相距 1、2、4、... (2 的幂)。在我们的例子中， $XY[14]$ 需要 $\log_2(16) - 1 = 3$ 个部分和，这些位置分别是 1 (在 $XY[14]$ 和 $XY[13]$ 之间)、2 (在 $XY[13]$ 和 $XY[11]$ 之间) 和 4 (在 $XY[11]$ 和 $XY[7]$ 之间)。

为了组织加法操作的后半部分，我们将首先展示所有需要从四个位置之外的部分和的操作，然后是从两个位置之外的操作，最后是从一个位置之外的操作。在反向树的第一级中，我们将 $XY[7]$ 加到 $XY[11]$ ，使得 $XY[11]$ 达到最终结果。在图 11.6 中，位置 11 是唯一达到最终结果的位置。在第二级中，我们完成了 $XY[5]$ 、 $XY[9]$ 和 $XY[13]$ ，它们可以通过来自两个位置之外的部分和分别完成： $XY[3]$ 、 $XY[7]$ 和 $XY[11]$ 。最后，在第三级中，我们通过累积来自一个位置之外的部分和（每个位置的左邻位置）完成所有偶数位置 $XY[2]$ 、 $XY[4]$ 、 $XY[6]$ 、 $XY[8]$ 、 $XY[10]$ 和 $XY[12]$ 。

现在我们准备实现 Brent-Kung 方法的扫描算法。我们可以通过以下循环来实现并行扫描的归约树阶段：

```
for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
    __syncthreads();
    if ((threadIdx.x + 1) % (2 * stride) == 0) {
        XY[threadIdx.x] += XY[threadIdx.x - stride];
    }
}
```

请注意，这个循环与图 10.6 中的归约类似。只有两个区别。第一个区别是我们将和值累积到最高位置，即 $XY[\text{blockDim.x} - 1]$ ，而不是 $XY[0]$ 。这是因为最高位置的最终结果是总和。因此，每个活动线程通过从其索引中减去步幅值来获取左侧的部分和。第二个区别是我们希望活动线程的线程索引形式为 $2n - 1$ ，而不是 $2n$ 。这就是为什么在选择执行每次迭代加法操作的线程时，我们在 modulo (%) 操作之前将 threadIdx.x 加 1。

这种归约方式的一个缺点是它存在显著的控制分歧问题。正如我们在第 10 章《归约和最小化分歧》中所看到的，更好的方法是使用不断减少的连续线程来执行加法操作。然而，不幸的是，我们在图 10.8 中用来减少分歧的技术不能在扫描归约树阶段使用，因为它不会在中间的 XY 位置生成所需的部分和值。因此，我们采用更复杂的线程索引到数据索引的映射，这将连续的线程部分映射到相距步幅距离的一系列数据位置。以下代码通过将连续的线程部分映射到索引形式为 $k * 2^n - 1$ 的 XY 位置来实现这一点：

```
for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
    __syncthreads();
    int index = (threadIdx.x + 1) * 2 * stride - 1;
    if(index < SECTION_SIZE) {
        XY[index] += XY[index - stride];
    }
}
```

通过在每次 for 循环迭代中使用这种复杂的索引计算，一组从线程 0 开始的连续线程将在每次迭代中使用，以避免 warp 内的控制分歧。在图 11.5 中的小例子中，块中有 16 个线程。在第一次迭代中，步幅等于 1。块中的前八个连续线程将满足 if 条件。为这些线程计算的 XY 索引值将是 1、3、5、7、9、11、13 和 15。这些线程将执行图 11.5 中的

第一行加法。在第二次迭代中，步幅等于2。块中的前四个线程将满足if条件。为这些线程计算的索引值将是3、7、11和15。这些线程将执行图11.5中的第二行加法。注意，由于每次迭代总是使用连续线程，因此控制分歧问题直到活跃线程数量低于warp大小时才会出现。

反向树的实现稍微复杂一些。我们看到步幅值从SECTION_SIZE/4减小到1。在每次迭代中，我们需要将XY元素的值从两倍步幅值减1的位置“推”到右边步幅位置。例如，在图11.5中，步幅值从4 (2^2) 减小到1。在第一次迭代中，我们希望将XY[7]的值推（加）到XY[11]，其中7是 $2 * 2^2 - 1$ ，距离（步幅）是 2^2 。注意，在此迭代中仅使用线程0，因为其他线程计算的索引太大，无法满足if条件。在第二次迭代中，我们将XY[3]、XY[7]和XY[11]的值推到XY[5]、XY[9]和XY[13]，分别对应于3、7和11是 $1 * 2 * 2^1 - 1$ 、 $2 * 2 * 2^1 - 1$ 和 $3 * 2 * 2^1 - 1$ 。目的位置距源位置 2^1 步幅距离。最后，在第三次迭代中，我们将所有奇数位置的值推到其右侧的偶数位置（步幅= 2^0 ）。

基于上述讨论，可以使用以下循环实现反向树：

```
for (int stride = SECTION_SIZE/4; stride > 0; stride /= 2) {
    __syncthreads();
    int index = (threadIdx.x + 1)*stride*2 - 1;
    if(index + stride < SECTION_SIZE) {
        XY[index + stride] += XY[index];
    }
}
```

索引的计算类似于在归约树阶段的计算。XY[index+stride] += XY[index]语句反映了从线程映射位置推到步幅距离较高位置的操作。图11.7展示了Brent-Kung并行扫描的最终内核代码。读者应该注意到，无论是归约阶段还是分发阶段，我们都不需要超过SECTION_SIZE/2的线程。因此，我们可以简单地在块中启动一个包含SECTION_SIZE/2线程的内核。由于一个块中最多可以有1024个线程，每个扫描区段最多可以包含2048个元素。然而，我们需要在开始时让每个线程加载两个X元素，并在结束时存储两个Y元素。

```
01  __global__ void Brent_Kung_scan_kernel(float *X, float *Y, unsigned int N) {
02      __shared__ float XY[SECTION_SIZE];
03      unsigned int i = 2*blockIdx.x*blockDim.x + threadIdx.x;
04      if(i < N) XY[threadIdx.x] = X[i];
05      if(i + blockDim.x < N) XY[threadIdx.x + blockDim.x] = X[i + blockDim.x];
06      for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
07          __syncthreads();
08          unsigned int index = (threadIdx.x + 1)*2*stride - 1;
09          if(index < SECTION_SIZE) {
10              XY[index] += XY[index - stride];
11          }
12      }
13      for (int stride = SECTION_SIZE/4; stride > 0; stride /= 2) {
14          __syncthreads();
15          unsigned int index = (threadIdx.x + 1)*stride*2 - 1;
16          if(index + stride < SECTION_SIZE) {
17              XY[index + stride] += XY[index];
18          }
19      }
20      __syncthreads();
21      if (i < N) Y[i] = XY[threadIdx.x];
22      if (i + blockDim.x < N) Y[i + blockDim.x] = XY[threadIdx.x + blockDim.x];
23  }
```

图11.7 Brent-Kung算法的包含（分段）扫描内核。

正如在Kogge-Stone扫描内核中一样，可以通过对加载X元素到XY语句进行微调，将Brent-Kung包含并行扫描内核轻松调整为排他扫描内核。感兴趣的读者还应该阅读Harris等人2007年的论文，该论文介绍了一种基于不同

反向树阶段设计的本地排他扫描内核。

现在我们转向反向树阶段操作次数的分析。操作次数是 $(16/8) - 1 + (16/4) - 1 + (16/2) - 1$ 。一般来说，对于 N 个输入元素，总操作次数为 $(2 - 1) + (4 - 1) + \dots + (N/4 - 1) + (N/2 - 1)$ ，即 $N - 1 - \log_2(N)$ 。因此，包括归约树($N - 1$ 次操作)和反向树($N - 1 - \log_2(N)$ 次操作)阶段在内的并行扫描总操作次数为 $2N - 2 - \log_2(N)$ 。注意，总操作次数现在是 $O(N)$ ，而Kogge-Stone算法为 $O(N * \log_2(N))$ 。

Brent-Kung算法相较于Kogge-Stone算法的优势非常明显。随着输入段变大，Brent-Kung算法执行的操作次数从未超过顺序算法的两倍。在受限能量的执行环境中，Brent-Kung算法在并行性和效率之间达到了良好的平衡。

虽然Brent-Kung算法在理论工作效率上远高于Kogge-Stone算法，但在CUDA内核实现中的优势较为有限。请记住，Brent-Kung算法使用的是 $N/2$ 线程。主要区别在于，在归约树阶段，活跃线程数量比Kogge-Stone算法下降得更快。然而，一些非活跃线程可能仍然消耗CUDA设备中的执行资源，因为它们与其他活跃线程通过SIMD绑定。这使得Brent-Kung相较于Kogge-Stone的工作效率优势在CUDA设备上不那么显著。

Brent-Kung算法相较于Kogge-Stone算法的主要劣势在于其潜在的更长执行时间，尽管其工作效率更高。在无限执行资源的情况下，Brent-Kung由于需要执行额外的反向树阶段步骤，可能需要大约两倍于Kogge-Stone的时间。然而，当我们有有限的执行资源时，速度比较可能会非常不同。使用11.3节中的例子，如果我们使用32个执行单元处理1024个输入元素，预计Brent-Kung内核大约需要 $(2 * 1024 - 2 - 10) / 32 = 63.6$ 步。读者应验证，当每个阶段活跃线程数量下降到32以下时，控制分歧将导致大约增加五步。这将导致相较于顺序执行的14倍加速 ($1024 / 73.6 = 14$)。与Kogge-Stone的320时间单位和3.2倍加速相比，当有更多执行资源和/或更长的延迟时，比较结果将更有利于Kogge-Stone。

11.5 为了提高工作效率的粗化方法

在多个线程中并行化扫描的开销类似于归约操作，它包括树执行模式的硬件未充分利用和同步开销。然而，扫描还有一个额外的并行化开销，即工作效率的降低。正如我们所见，并行扫描的工作效率低于顺序扫描。如果线程真正能够并行运行，那么这种较低的工作效率是可以接受的代价。然而，如果硬件将其串行化，我们最好通过线程粗化自行串行化，以提高工作效率。

我们可以通过在输入的子段上添加一个完全独立的顺序扫描阶段，设计出一个工作效率更高的并行分段扫描算法。每个线程块接收的输入段大于原始段的粗化因子。在算法开始时，我们将块的输入段划分为多个连续的子段：每个线程对应一个子段。子段的数量与线程块中的线程数量相同。

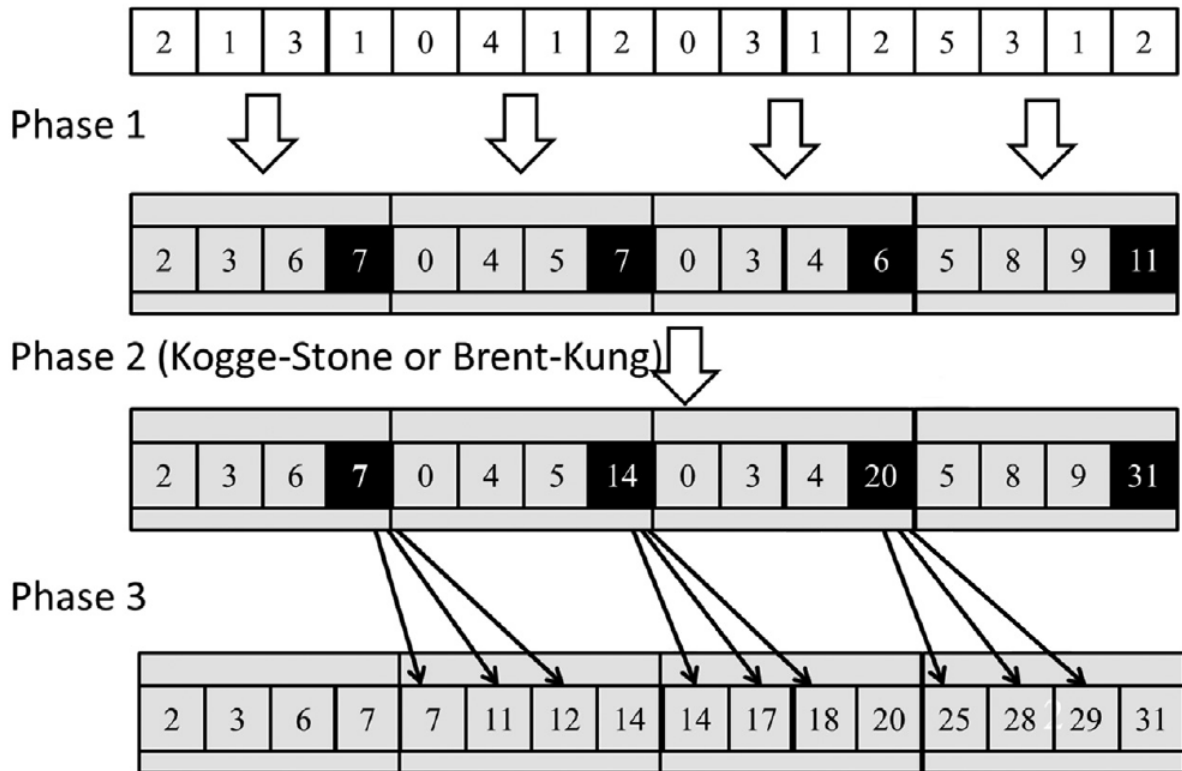


图11.8 为了提高工作效率的三阶段并行扫描。

粗化扫描分为三个阶段，如图11.8所示。在第一阶段，我们让每个线程对其连续子段执行顺序扫描。例如，在图11.8中，我们假设一个块中有四个线程。我们将16个元素的输入段划分为四个子段，每个子段包含四个元素。线程0将对其子段(2, 1, 3, 1)进行扫描并生成(2, 3, 6, 7)。线程1将对其子段(0, 4, 1, 2)进行扫描并生成(0, 4, 5, 7)，依此类推。

注意，如果每个线程直接从全局内存访问输入进行扫描，其访问将不会合并。例如，在第一次迭代中，线程0将访问输入元素0，线程1将访问输入元素4，依此类推。因此，我们通过使用共享内存吸收无法合并的内存访问来改善内存合并，如第6章“性能考虑”中所述。也就是说，我们以合并的方式在共享内存和全局内存之间传输数据，并在共享内存中执行不利的访问模式。在第二阶段开始时，所有线程协作以迭代方式将输入加载到共享内存中。在每次迭代中，相邻线程加载相邻元素以实现内存合并。例如，在图11.8中，所有线程协作并以合并的方式加载四个元素：线程0加载元素0，线程1加载元素1，依此类推。然后，所有线程继续加载下四个元素：线程0加载元素4，线程1加载元素5，依此类推。

一旦所有输入元素都在共享内存中，线程就从共享内存中访问它们自己的子段并对其执行顺序扫描。这显示为图11.8中的阶段1。注意，在阶段1结束时，每个子段的最后一个元素（在第二行中以黑色突出显示）包含该子段中所有输入元素的和。例如，第0段的最后一个元素包含值7，即该段中输入元素(2, 1, 3, 1)的和。

在第二阶段，每个块中的所有线程协作并在由每个子段的最后一个元素组成的逻辑数组上执行扫描操作。由于元素数量有限（等于块中的线程数量），可以使用Kogge-Stone或Brent-Kung算法执行这一步骤。注意，由于需要扫描的元素距离（在图11.8中为四个元素）较远，线程到元素的映射需要稍作修改，与图11.3和图11.7中的映射不同。

在第三阶段，每个线程将前一个子段最后一个元素的新值加到其元素上。在此阶段，不需要更新每个子段的最后一个元素。例如，在图11.8中，线程1将值7加到其子段中的元素(0, 4, 5)上，生成(7, 11, 12)。该子段的最后一个元素已经是正确的值14，不需要更新。

通过这种三阶段方法，我们可以使用比一个段中元素数量少得多的线程。段的最大尺寸不再受限于块中线程数量，而是受限于共享内存的大小；段中的所有元素都需要适应共享内存。线程粗化对扫描的主要优势在于其高

效利用执行资源。假设我们在第二阶段使用Kogge-Stone算法。对于 N 个元素的输入列表，如果我们使用 T 个线程，每个阶段的工作量分别为 $N - T$ （第一阶段）、 $T * \log_2 T$ （第二阶段）和 $N - T$ （第三阶段）。如果我们使用 P 个执行单元，可以预计执行将需要 $(N - T + T * \log_2 T + N - T) / P$ 步。例如，如果我们使用64个线程和32个执行单元处理1024个元素，算法大约需要 $(1024 - 64 + 64 * 6 + 1024 - 64) / 32 = 72$ 步。我们将粗化扫描内核的实现留作读者的练习。

11.6 针对任意长度输入的分段并行扫描

对于许多应用来说，需要处理的扫描操作的元素数量可能达到数百万甚至数十亿。我们之前介绍的内核执行的是局部块范围内的输入段扫描，但我们仍然需要一种方法来整合不同段的结果。为此，我们可以使用分层扫描方法，如图11.9所示。

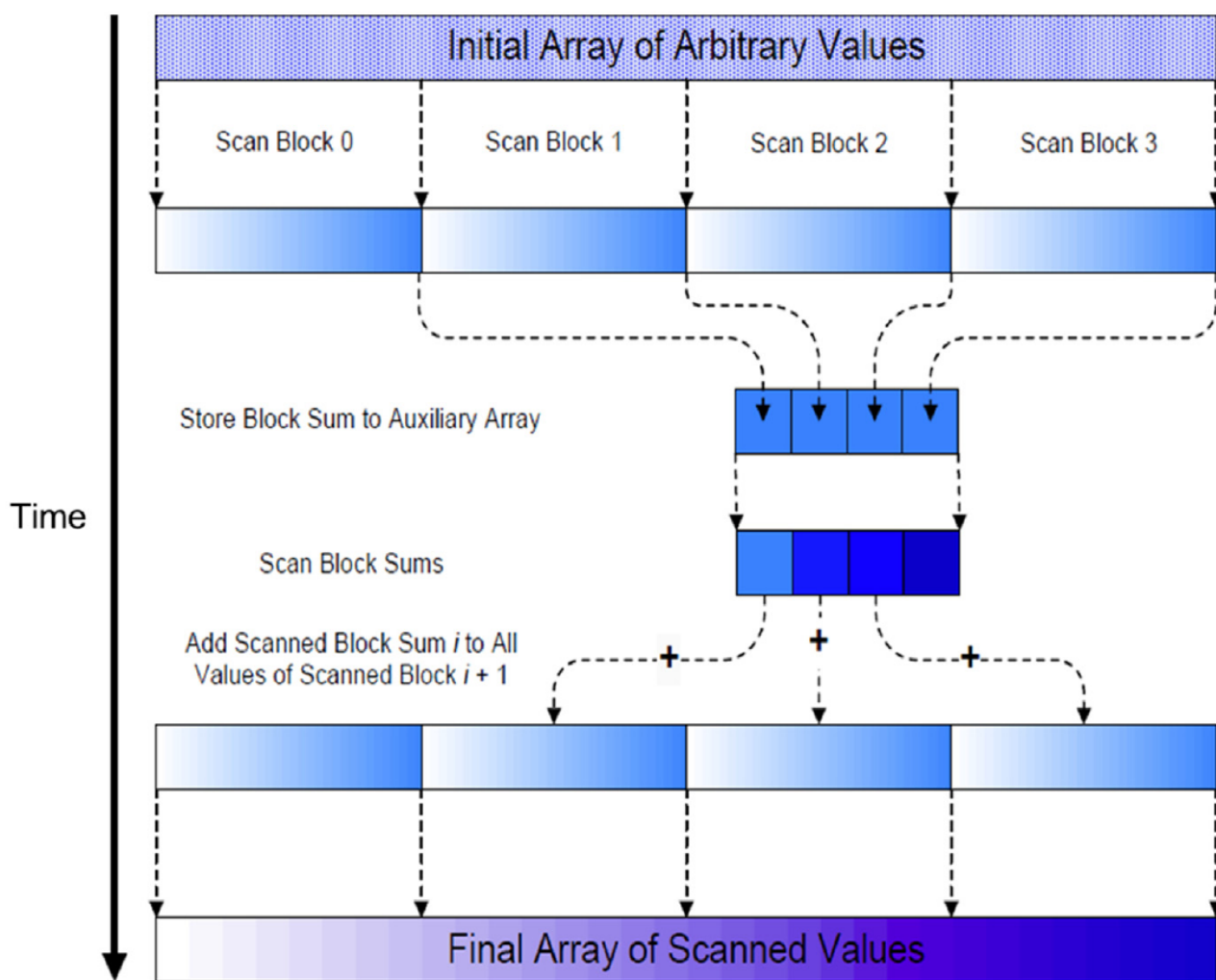


图11.9 针对任意长度输入的分层扫描。

对于一个大型数据集，我们首先将输入划分为各个段，使得每个段都能适合一个流多处理器的共享内存，并由单个块处理。假设我们对一个大型输入数据集调用图11.3和11.7中的某一个内核。在网格执行结束时， Y 数组将包含各个段的扫描结果，如图11.9所示称为扫描块。每个扫描块中的每个元素仅包含相同扫描块中所有前面元素的累积值。这些扫描块需要组合成最终结果；也就是说，我们需要调用另一个内核，将所有前面扫描块中的元素总和加到每个扫描块的每个元素上。

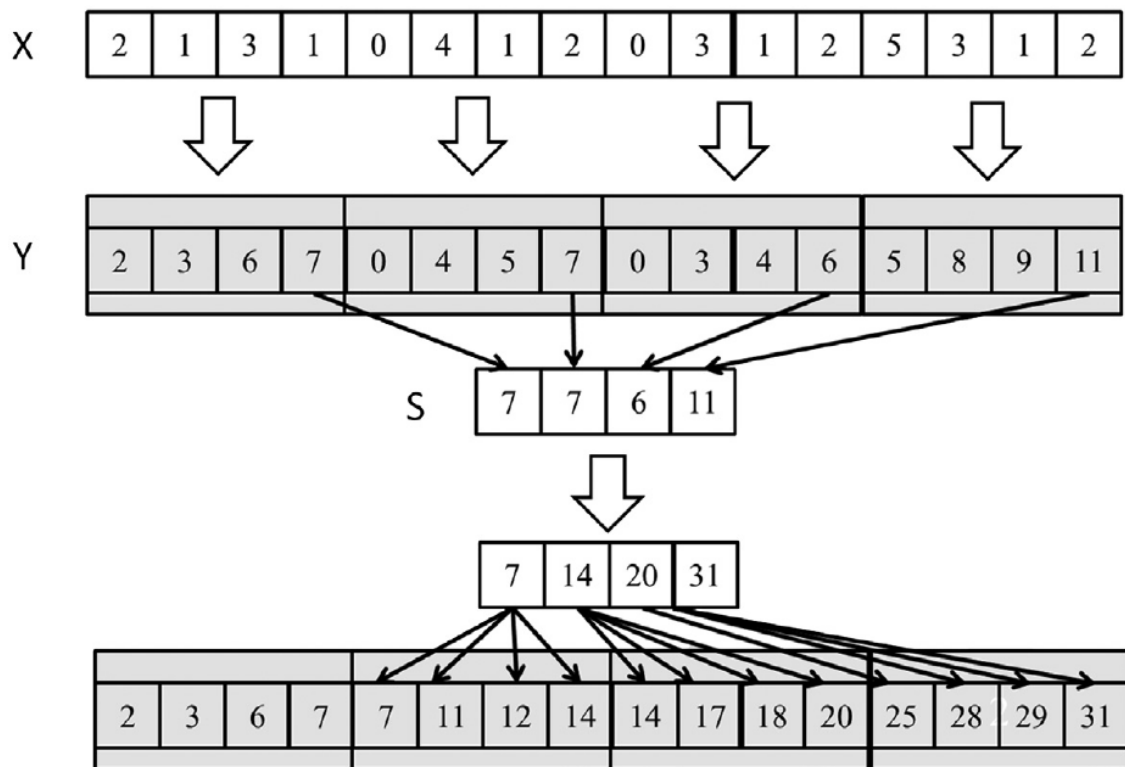


图11.10 分层扫描示例

图11.10展示了图11.9的分层扫描方法的一个小例子。在这个例子中，有16个输入元素被划分为四个扫描块。我们可以使用Kogge-Stone内核、Brent-Kung内核或一个粗化的内核来处理各个扫描块。内核将这四个扫描块视为独立的输入数据集。在扫描内核终止后，每个Y元素包含其扫描块内的扫描结果。例如，扫描块1的输入为0、4、1、2。扫描内核生成该段的扫描结果，即0、4、5、7。注意，这些结果不包含扫描块0中任何元素的贡献。为了生成该扫描块的最终结果，应将扫描块0中所有元素的总和，即 $2+1+3+1=7$ ，加到扫描块1的每个结果元素上。再举一个例子，扫描块2的输入是0、3、1、2。内核生成该扫描块的扫描结果，即0、3、4、6。为了生成该扫描块的最终结果，应将扫描块0和扫描块1中所有元素的总和，即 $2+1+3+1+0+4+1+2=14$ ，加到扫描块2的每个结果元素上。

重要的是，每个扫描块的最后一个输出元素给出了该扫描块所有输入元素的总和。这些值在图11.10中分别是7、7、6和11。这将我们带到了图11.9中分段扫描算法的第二步，即收集每个扫描块的最后结果元素到一个数组中，并对这些输出元素进行扫描。这一步也在图11.10中有所展示，所有扫描块的最后扫描输出元素被收集到一个新的数组S中。虽然图11.10的第二步在逻辑上与图11.8的第二步相同，但主要区别在于图11.10涉及不同线程块的线程。因此，每个段的最后一个元素需要被收集（写入）到一个全局内存数组中，以便它们在线程块之间可见。

通过更改扫描内核结束时的代码，可以收集每个扫描块的最后结果，使得每个块的最后一个线程使用其 `blockIdx.x` 作为数组索引将其结果写入S数组。然后对S进行扫描操作以生成输出值7、14、20、31。注意，这些第二级扫描输出值是从起始位置X[0]到每个扫描块末端位置的累积总和。即， $S[0]=7$ 的值是从X[0]到扫描块0末端位置X[3]的累积总和。 $S[1]=14$ 的值是从X[0]到扫描块1末端位置X[7]的累积总和。因此，S数组中的输出值在原始扫描问题的“战略”位置给出了扫描结果。换句话说，在图11.10中，S[0]、S[1]、S[2]和S[3]中的输出值分别在位置X[3]、X[7]、X[11]和X[15]给出了原始问题的最终扫描结果。这些结果可用于将每个扫描块中的部分结果变为最终值。

这将引出图 11.10 中分段扫描算法的最后一步。二级扫描输出值被添加到它们对应的扫描块的值中。例如，在图 11.10 中，S[0] 的值（值为 7）将被添加到线程块 1 的 Y[0]、Y[1]、Y[2] 和 Y[3] 上，这完成了这些位置的结果。这些位置的最终结果是 7、11、12 和 14。这是因为 S[0] 包含了原始输入 X[0] 到 X[3] 的值的总和。这些最终

结果是 14、17、18 和 20。 $S[1]$ 的值 (14) 将被添加到 $Y[8]$ 、 $Y[9]$ 、 $Y[10]$ 和 $Y[11]$ 上，这完成了这些位置的结果。 $S[2]$ 的值 (20) 将被添加到 $Y[12]$ 、 $Y[13]$ 、 $Y[14]$ 和 $Y[15]$ 上。最后， $S[3]$ 的值是原始输入所有元素的总和，也是 $Y[15]$ 的最终结果。

熟悉计算机算术算法的读者应该认识到，分段扫描算法背后的原理与现代处理器的硬件加法器中的进位预测原理非常相似。考虑到我们迄今为止研究的两种并行扫描算法也是基于创新的硬件加法器设计，这并不奇怪。

我们可以使用三个内核来实现分段扫描。第一个内核在很大程度上与三阶段内核相同（我们同样可以使用 Kogge-Stone 内核或 Brent-Kung 内核）。我们需要添加一个更多的参数 S ，其维度为 $N/SECTION_SIZE$ 。在内核的末尾，我们为块中最后一个线程添加一个条件语句，以将扫描块中最后一个 XY 元素的输出值写入 S 的 $blockIdx.x$ 位置：

```
__syncthreads();
if (threadIdx.x == blockDim.x - 1) {
    S[blockIdx.x] = XY[SECTION_SIZE - 1];
}
```

第二个内核只是配置为单线程块的三个并行扫描内核之一，它将 S 作为输入并将 S 作为输出，而不产生任何部分和。

第三个内核将 S 数组和 Y 数组作为输入，并将其输出写回 Y 。假设我们在每个块中启动 $SECTION_SIZE$ 线程，每个线程将一个 S 元素（由 $blockIdx.x-1$ 选择）添加到一个 Y 元素：

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
Y[i] += S[blockIdx.x - 1];
```

换句话说，块中的线程将所有先前扫描块的总和添加到它们的扫描块的元素中。我们留给读者作为练习完成每个内核的细节并完成主机代码。

11.7 提高内存访问效率的单次扫描

在第 11.6 节中提到的分段扫描中，部分扫描结果（扫描块）在启动全局扫描内核之前被存储到全局内存中，然后第三个内核从全局内存中重新加载这些结果。执行这些额外的内存存储和加载的时间与后续内核中的计算不重叠，这可能显著影响分段扫描算法的速度。为了避免这种负面影响，已经提出了多种技术（Dotsenko 等人，2008；Merrill 和 Garland，2016；Yan 等人，2013）。在本章中讨论了一种基于流的扫描算法。鼓励读者阅读参考文献以了解其他技术。

在 CUDA C 编程的背景下，基于流的扫描算法（不要与将在第 20 章“异构计算集群编程”中介绍的 CUDA 流混淆），或称为多米诺式扫描算法，指的是一种分段扫描算法，其中部分和数据在同一个网格中的相邻线程块之间通过全局内存单向传递。基于流的扫描算法建立在一个关键观察之上，即全局扫描步骤（图 11.9 的中间部分）可以以多米诺的方式执行，并且不真正需要网格范围的同步。例如，在图 11.10 中，扫描块 0 可以将其部分和值 7 传递给扫描块 1 并完成其任务。扫描块 1 接收来自扫描块 0 的部分和值 7，与其局部部分和值 7 相加得到 14，将其部分和值 14 传递给扫描块 2，然后通过将 7 添加到其扫描块中的所有部分扫描值来完成其最终步骤。这一过程在所有线程块中持续进行。

为了实现多米诺式扫描算法，可以编写一个内核来执行图 11.9 中分段扫描算法的所有三个步骤。线程块 i 首先对其扫描块执行扫描，使用我们在第 11.2 至 11.5 节中介绍的三种并行算法之一。然后它等待其左侧邻居块 $i-$

1 传递和值。一旦它收到来自块 $i - 1$ 的和值，它将该值添加到其局部和并将累计和值传递给其右侧邻居块 $i + 1$ 。然后它继续将从块 $i - 1$ 接收到的和值添加到所有部分扫描值中，以生成扫描块的所有输出值。

在内核的第一阶段，所有块都可以并行执行。在数据传递阶段，它们将被串行化。然而，一旦每个块收到其前任块的和值，它可以在与所有其他已收到其前任块的和值的块并行的情况下执行其最终阶段。只要和值能快速通过各块传递，在第三阶段块之间可以有充足的并行性。

为了使这种多米诺式扫描工作，需要相邻（块）同步（Yan 等人，2013）。相邻(adjacent)同步是一种定制的同步方法，允许相邻线程块同步和/或交换数据。特别是在扫描中，数据从扫描块 $i - 1$ 传递到扫描块 i ，像一个生产者-消费者链。在生产者侧（扫描块 $i - 1$ ），在部分和被存储到内存后一个标志被设置为特定值，而在消费者侧（扫描块 i ），检查该标志是否为该特定值，然后再加载传递的部分和。如前所述，加载的值进一步与局部和相加，然后传递到下一个块（扫描块 $i + 1$ ）。相邻同步可以通过使用原子操作来实现。以下代码段展示了使用原子操作实现相邻同步：

```
__shared__ float previous_sum;
if (threadIdx.x == 0){
    // Wait for previous flag
    while(atomicAdd(&flags[bid], 0) == 0) { }
    // Read previous partial sum
    previous_sum = scan_value[bid];
    // Propagate partial sum
    scan_value[bid + 1] = previous_sum + local_sum;
    // Memory fence
    __threadfence();
    // Set flag
    atomicAdd(&flags[bid + 1], 1);
}
__syncthreads();
```

这段代码由每个块中的一个领导线程（例如，索引为0的线程）执行。其余线程将在最后一行的__syncthreads()处等待。在块bid中，领导线程反复检查全局内存数组flags[bid]，直到其被设置为止。然后，它通过访问全局内存数组scan_value[bid]加载其前驱的部分和，并将该值存储到其本地共享内存变量previous_sum中。它将previous_sum与其本地部分和local_sum相加，并将结果存储到全局内存数组scan_value[bid+1]中。内存栅栏函数__threadfence()用于确保scan_value[bid+1]值在将flag设置为atomicAdd()之前到达全局内存。

尽管在flags数组上的原子操作和对scan_value数组的访问可能会产生全局内存流量，但这些操作主要是在最近GPU架构的第二级缓存中执行（第9章，并行直方图）。任何这样的全局内存存储和加载都可能与其他块的第1和第3阶段计算活动重叠。另一方面，在执行第11.5节中的三个内核分段扫描算法时，在全局内存中对S数组的存储和加载是在一个单独的内核中进行的，并且不能与第1或第3阶段重叠。

多米诺式算法存在一个微妙的问题。在GPU中，线程块可能并不总是按照其blockIdx值线性调度，这意味着扫描块i可能在扫描块i + 1之后被调度和执行。在这种情况下，调度器安排的执行顺序可能与相邻同步代码所假定的执行顺序相矛盾，导致性能降低甚至死锁。例如，调度器可能会在调度扫描块i - 1之前，将扫描块i通过扫描块i + N进行调度。如果扫描块i到扫描块i + N占用了所有流处理器，那么扫描块i - 1将无法开始执行，直到其中至少一个完成执行。然而，它们都在等待来自扫描块i - 1的和值。这导致系统死锁。

有多种技术可以解决这个问题 (Gupta等人, 2012; Yan等人, 2013)。在这里, 我们只讨论一种特定的方法, 即动态块索引分配, 并将其余内容作为读者的参考。动态块索引分配将线程块索引的分配与内置的`blockIdx.x`分离。在单次扫描中, 每个块的`bid`变量的值不再与`blockIdx.x`的值相关联。相反, 它是通过在内核开始时使用以下代码确定的:

```
__shared__ unsigned int bid_s;
if (threadIdx.x == 0) {
    bid_s = atomicAdd(blockCounter, 1);
}
__syncthreads();
unsigned int bid = bid_s;
```

领导线程原子地增加一个由`blockCounter`指向的全局计数器变量。全局计数器存储下一个被调度的块的动态块索引。然后, 领导线程将获得的动态块索引值存储到一个共享内存变量`bid_s`中, 以便在`__syncthreads()`之后所有线程都可以访问它。这确保所有扫描块都是线性调度的, 并防止潜在的死锁。换句话说, 如果一个块获得了一个值为`i`的`bid`值, 则可以保证一个值为`i - 1`的块已经被调度, 因为它已经执行了原子操作。

11.8 总结

这一章我们学习了并行扫描, 也称为前缀和, 作为一个重要的并行计算模式。扫描被用来实现对需求不均匀的各方进行并行资源分配。它将基于数学递归的看似顺序计算转换为并行计算, 有助于减少许多应用程序中的顺序瓶颈。我们展示了一个简单的顺序扫描算法对`N`个元素的输入只进行了`N-1`次或者 $O(N)$ 次加法运算。

首先介绍了一个快速且概念简单但工作效率不高的Kogge-Stone分段扫描算法。该算法执行 $O(N \log_2 N)$ 次操作, 比其顺序对应物要多。随着数据集的大小增加, 为了使并行算法与简单的顺序算法达到平衡所需的执行单元数量也会增加。因此, Kogge-Stone扫描算法通常用于处理具有丰富执行资源的处理器中的适度大小的扫描块。

然后介绍了一个概念上更复杂的Brent-Kung分段扫描算法。该算法通过使用约简树阶段和反向树阶段, 在不论输入数据集有多大的情况下只执行 $2 \cdot N - 3$ 次或者 $O(N)$ 次加法运算。这样的工作效率算法, 其操作数量随输入集大小线性增长, 通常也被称为数据可扩展算法。虽然Brent-Kung算法比Kogge-Stone算法具有更好的工作效率, 但它需要更多的步骤来完成。因此, 在具有足够执行资源的系统中, 预计Kogge-Stone算法将具有更好的性能, 尽管工作效率较低。

我们还应用了线程粗化来减轻并行扫描的硬件利用率和同步开销, 并提高其工作效率。线程粗化是通过使块中的每个线程在合作执行不太高效的块级并行扫描生成整个块部分之前, 对其自己的输入元素的一个子部分进行高效的顺序扫描来实现的。

我们提出了一种层次扫描方法, 将并行扫描算法扩展到处理任意大小的输入集。不幸的是, 分段扫描算法的直接、三个内核实现会导致冗余的全局内存访问, 其延迟与计算不重叠。因此, 我们还提出了一种多米诺式的层次扫描算法, 以实现单次、单内核的实现, 并提高层次扫描算法的全局内存访问效率。然而, 这种方法需要使用原子操作、线程内存栅栏和屏障同步来设计精心的相邻块同步机制。还必须特别注意防止死锁, 通过使用动态块索引分配。

还有更多优化机会可以实现更高性能的实现, 例如使用warp级别的洗牌操作。总的来说, 在GPU上实现和优化并行扫描算法是复杂的过程, 一般用户更有可能使用GPU上的并行扫描库, 如Thrust (Bell and Hoberock,

2012)，而不是从头开始实现自己的扫描内核。尽管如此，并行扫描是一个重要的并行模式，它提供了一个有趣且相关的案例研究，展示了优化并行模式所涉及的权衡考虑。